Simulating a Particle Detector and Particle Decay in C++

Aidan Hughes

11306492

School of Physics and Astronomy

University of Manchester

Third Year Code Report

May 2026

Abstract

The aim of this project was to utilize object oriented programming practices to create a particle detector and particle decay simulators in C++. It consists of two main inheritance chains, one for the particle classes and another for sub-detector classes. These classes, and their respective container classes ParticleBunch and Detector, practice encapsulation by storing data members as private or protected with access via public member functions. The use of virtual and pure virtual functions in the abstract parent classes of the hierarchy enables polymorphism in the child classes.

1 Introduction

The aim of this project was to create a C++ library that simulates a particle detector and its detecting functionality through the use of object oriented programming. As an addition, this code also aims to simulate particle decays within an accelerator. Particle detectors, such as ATLAS or the CMS, compose of multiple sub-detector types, each of which measure different types of particles.
The tracker, the first sub-detector a particle will pass through, uses magnetic fields to measure particle paths, so can only measure charged particles. Next are the calorimeters, which are split into two, an electromagnetic and a hadronic calorimeter. The electromagnetic calorimeter causes electrons and photons to trigger a particle shower which is measured to finds their energy. The hadronic calorimeter does the same but for protons and neutrons. Finally, due to muons high energy, they are not turned into showers by the calorimeters, and pass through to the Muon Spectrometer sub-detector. Using information about which sub-detectors were triggered, the type of particle that passed through can be deduced. These sub-detectors also have a detection efficiency associated with them, which compares the number of particles detected to the number produced during an event, usually represented as a percentage.
Along with the particles mentioned above, tau and Higgs particles are also found in accelerators, but decay quickly to other particles. This project focuses on the decays to muon and neutrinos for tau and di-photon decay for the Higgs. All particles have a four-momentum, which is a relativistically conserved property, akin to conservation of three-momentum in classical mechanics. Many particles also have masses, charges or even type specific conservation numbers like lepton number.
To simulate the sub-detectors and particles in C++, an inheritance chain was established for each.

2 Code Implementation

Object oriented programming (OOP) is built on four pillars: encapsulation, inheritance, polymorphism and abstraction.

2.1 FourMomentum

The FourMomentum class is constructed from 4 doubles corresponding to each four-momentum component: energy and the x, y and z momenta. The four-momentum components were stored as private double member variables within the FourMomentum class, as this is a more readable format then a vector in which the index corresponding to each property is not explicit to anyone extending the class. The components are accessed via getters and setters, returning the value as a double, and the setters taking a double as input and changing the member variable. Using getters and setters along with private variables is an example of encapsulation. In this case the encapsulation allowed for value checking on inputted energy, Listing 1, to ensure it was positive,

throwing an error if not, which would not have been possible if the data member was publicly accessible.

```
30 // Setter
31 void FourMomentum::set_energy(double energy)
32 {
33   if(!validate_energy(energy))
34   {
35     throw std::invalid_argument("negative energy");
36   }
37   energy_ = energy;
38 }
```

Listing 1: Showing setter for energy private data member in FourMomentum.cpp.

A major benefit of constructing separate class to hold particle four momentum is to be able to overload operators, such as the addition (+) and increment (+=) operators shown in Listing 2, to provide functions more suitable to the class. The addition operator now adds two four-momenta component-wise and returns the result as a new FourMomentum object, acting as a form of static (compile-time) polymorphism. The advantage of this can be seen in the overloaded increment operator, where the addition operator is used to add the second FourMomentum object's components into the first's, utilizing the 'this' pointer, which always points to the calling object.

```
39 // Addition
40 FourMomentum FourMomentum::operator+(const FourMomentum &vector) const
41 {
42   FourMomentum result(energy_ + vector.energy_, x_momentum_
43     + vector.x_momentum_, y_momentum_ + vector.y_momentum_,
44                 z_momentum_ + vector.z_momentum_);
45   return result;
46 }
47 // Increment
48 FourMomentum & FourMomentum::operator+=(const FourMomentum &vector)
49 {
50   *this = *this + vector;
51   return *this;
52 }
```

Listing 2: Code snippet from FourMomentum.cpp showing the overloading of + and += operators to add FourMomentum objects component-wise.

Listing 2 also showcases the use of const and references, used widely in this project. First, the FourMomentum object, vector, is passed to the function as a const reference, which means a copy of memory address of the vector is passed to the function, rather than a copy of the object. This saves on memory usage as objects are much larger than their addresses, so take up more memory when copied. However, referencing allows the underlying object to be changed by the

function, which is not wanted for the addition and increment operators, so const is used, which ensures the object is not changed by the function using it. Moreover, the addition operator is defined as a const member function by the const at the end of line 40, which prevents it from changing the calling FourMomentum object. This is not included in the increment operator as that intents to change the calling FourMomentum, instead the return is a reference to the calling FourMomentum.
The FourMomentum class also includes dot product and info member functions, which return the dot product as a double or print the components of the FourMomentum.

2.2 Particle

The Particle class is constructed using a FourMomentum object, a double for mass and a integer for charge. The FourMomentum is stored as a unique pointer to the FourMomentum object, as this means only the address to the FourMomentum stored in the Particle. A smart pointer will protect against memory leaks by deleting the object and itself when it goes out of scope, unlike a raw pointer. The unique kind was chosen as only the particle can possess its own four momentum, so there is no need for other pointers to the FourMomentum, and therefore no need for a shared pointer.
A consequence of using smart pointers is the need to use the Rule of 5, that being the redefinition of the copy constructor and assignment, Listing 3, and the move constructor and assignment, for which the default was sufficient. The copy had to be constructor and assignment redefined as unique pointers cannot be copied or a new unique pointer made pointing to the same FourMomentum object as that would result in two unique pointers to the same object, which is not allowed as would result in the first pointer deleting itself and the object when out of scope, leaving the second pointing to nothing, therefore allowing memory leaks. So instead, a copy of the FourMomentum is made for the new (or existing for assignment) pointer to point to. The default move constructor and assignment can still used however, as moving a unique pointer does not cause two pointers to the object.

```
16 // Copy Constructor
17 Particle::Particle(const Particle &other_particle) : four_momentum_ptr_
18 (std::make_unique<FourMomentum>(*other_particle.four_momentum_ptr_)),
19 mass_(other_particle.mass_), charge_(other_particle.charge_){}
20 // Copy Assignment
21 Particle & Particle::operator=(const Particle &other_particle)
22 {
23   if(&other_particle == this){return *this;}
24
25   *four_momentum_ptr_ = *other_particle.four_momentum_ptr_;
26   mass_ = other_particle.mass_;
27   charge_ = other_particle.charge_;
28   return *this;
```

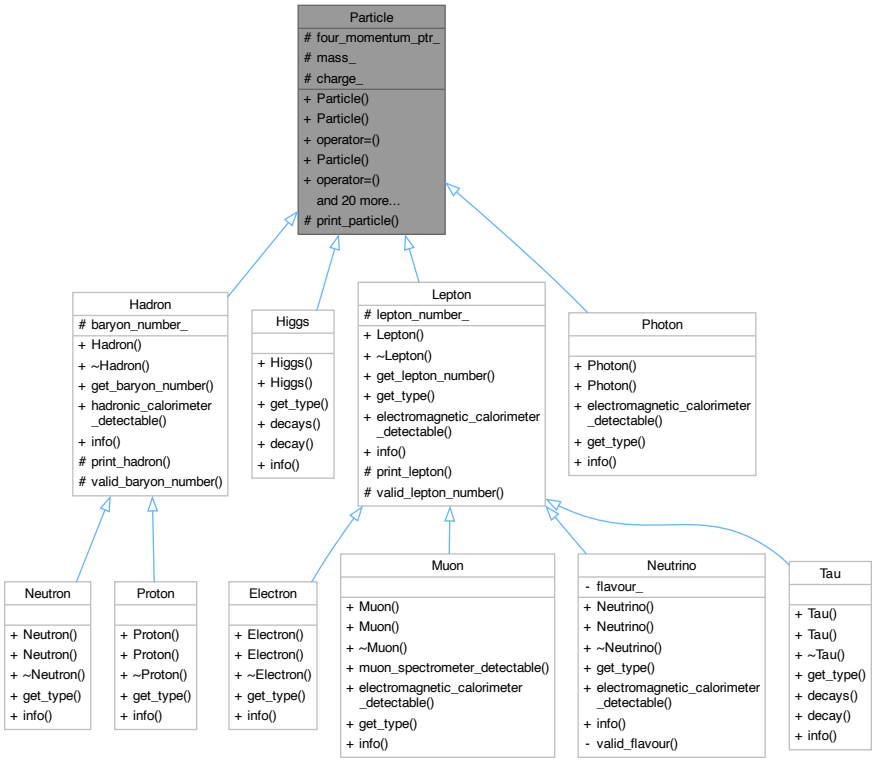

Figure 1: A UML class diagram for the Particle class inheritance hierarchy. Produced by Doxygen.

```
29 }
```

Listing 3: Code snippet from Particle.cpp showing the copy constructor and assignment redefinition in the Particle class.

2.3 Particle hierarchy

To create classes for the different types of particles, an inheritance hierarchy was used, shown in Figure 1. For inheritance, the destructors of the parent class must be defined and virtual, and destructors defined in child class. This is so only the child class destructor is called when the object goes out of scope, avoiding double deletion.

The Particle, Hadron and Lepton classes are abstract classes, so they cannot be instantiated, but allow child classes to inherit important data members and member functions from them. Abstract classes are created by making a function within them pure virtual, using virtual keyword and an =0 at the end of the function declaration. This function must then be defined in the child class, so the info function, which prints the data about an object in this project, was used as every class must have their own defined anyway.

Non pure virtual functions were also useful, especially for flags and the decay function. The flags were boolean functions that returned true if a specific property was true for that particle, useful if it couldn't be easily discerned from another property of the particle, for example when deciding if a particle could be detected by the electromagnetic calorimeter. This was used for dynamic polymorphism in the ParticleBunch and Detector classes.

```
16 // decay function - doesn't follow conservation laws.
17 std::deque<std::unique_ptr<Particle>> Tau::decay()
18 {
19   std::deque<std::unique_ptr<Particle>> decay_products{};
20   // splitting tau mass between two produced neutrinos
21   // producing muon decay product
22   decay_products.push_back(std::make_unique<Muon>(
23   Muon(std::move(*four_momentum_ptr_), std::move(lepton_number_))));
24   // producing neutrinos
25   // pushing neutrinos onto decay_products
26   return decay_products;
27 }
```

Listing 4: Code snippet from Tau.cpp decay member function. Neutrino instantiation and energy/momentum calculation omitted for clarity

The Higgs and Tau (and Particle) classes possess a decay function, which returns a deque of pointers to the decay products. The Tau decay to muon and neutrinos, Listing 4, showcases the move of FourMomentum to create new Muon with its properties.

2.4 ParticleBunch

The ParticleBunch class stored particles as a deque of base class unique pointers. Base class pointers allow all child classes of the base class to be stored in the same container via a unique pointer. Not only does this reduce the size of the deque stored on the object, it also allows for dynamic(run-time) polymorphism, shown in Listing 5 when particle ptr calls the decays function, which is a flag for if a particle decays.

```
108 void ParticleBunch::decay()
109 {
110   // deque to store undecayed particles and undecayed decay products
111   std::deque<std::unique_ptr<Particle>> result{};
112   while(!(particle_bunch_.empty()))
113   {
114     // moves first pointer to particle in deque
115     auto particle_ptr = std::move(particle_bunch_.front());
116     // removes front as now it is pointer to null
117     particle_bunch_.pop_front();
118     // checks if particle decays
119     if(particle_ptr->decays())
```

```
120     {
121       // decays particle and push decay products onto back of deque
122     }
123     else
124     {
125       // pushes undecayed particles into results
126       result.push_back(std::move(particle_ptr));
127     }
128   }
129   // restores particle bunch by moving results to it
130   particle_bunch_ = std::move(result);
131 }
```

Listing 5: Code snippet from Particle.cpp showing the copy constructor and assignment redefinition in the Particle class.

Listing 5 also shows the reason for using a deque to store the particle pointers, as a deque allows the particle next to be decayed to moved then popped from the front, whilst its decay products are pushed from the back of the container, resulting in first in first out. Therefore, if a decay product was decayable, it would also be decayed thanks to the while loop.
The ParticleBunch can be constructed from a vector of strings, which is the same way the main.cpp file reading function outputs the data read in. It then iterates over each line and uses a string stream to separate into data members, Listing 6, and converts string particle type name into the ParticleType enum class. After this, the data is sent to a switch-case, where the particle and its pointer are instantiated.

```
108 // converts data line into Particle base class pointer to be stored
109 std::unique_ptr<Particle> ParticleBunch::particle_from_file(
110         std::string data_line)
111 {
112   std::stringstream data{data_line};
113   // variable initialization
114   data>>type_name;
115   ParticleType type = string_to_particle_type(type_name);
116   // get neutrino flavor if neutrino
117   std::string flavour_name;
118   if(type == ParticleType::NEUTRINO|type == ParticleType::ANTINEUTRINO)
119   {
120     data>>flavour_name;
121   }
122   // gets remaining particle values
123   data>>energy>>x_momentum>>y_momentum>>z_momentum;
```

Listing 6: Code snippet of logic used to get particles from data file lines, variable initialization not included for clarity.

2.5 ParticleType

ParticleType is an enum class created to store all the particle types used, so they can be easily implemented as member variables, as in the case of the Neutrino flavour_, or to use switch case logic instead of overly long if else chains. An enum class was chosen over a normal enums so that when used as an input or stored variable, one cannot accidentally change or set a particle type using a number, which is possible with base enums.
The ParticleType.cpp also contains conversion functions for string to ParticleType and ParticleType to string, abstracting the validation of particle type inputs into one place. The string_to_particle_type function, Listing 7 , uses an unordered map to store key-value pairs, so the string can be looked up and the equivalent ParticleType returned. Using a map allows for multiple strings to correspond to the same ParticleType, such as with "positron" and "anti_electron" both converting to ParticleType::POSITRON.

```
13 // std::string to ParticleType
14 ParticleType string_to_particle_type(std::string name)
15 {
16 std::unordered_map<std::string, ParticleType> string_to_type{
17         {"electron", ParticleType::ELECTRON},
18         {"positron", ParticleType::POSITRON},
19         {"anti_electron", ParticleType::POSITRON},
20         // other particle types
21         };
22 auto iterator = string_to_type.find(name);
23 if(iterator == string_to_type.end())
24 {
25   throw std::invalid_argument("invalid particle type");
26 }
27 return iterator->second;
28 }
```

Listing 7: Code snippet from ParticleType.cpp shwoing the string_to_particle_type conversion function.

rand() for efficiency / misdetection calulation

2.6 SubDetector

```
36 bool SubDetector::misdetected() const
37 {
38   srand(time(0));
39   if (rand() % 100 >= efficiency_){return true;}
40   return false;
```

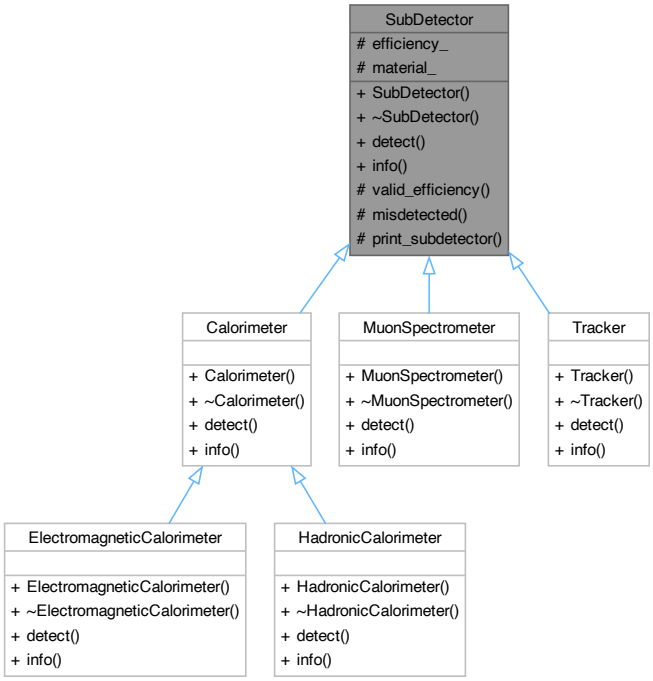

Figure 2: A UML class diagram for the Detector class inheritance hierarchy. Produced by Doxygen.

```
41 }
```

Listing 8: Code snippet misdetection function where rand() % 100 gets a number between 0-99, and if its greater than the efficiency, a misdetection occurs. srand(time(0)) seeds the randon number generation.

SubDetector has two protected member variables, efficiency, which is as a percentage and required to be between 1%-100%, and material which is a string. The efficiency is used in co-ordination with rand(), Listing to cause misdetections by returning true if one occurs, resulting in the detector returning 0.

2.7 SubDetector hierarchy

Figure 2 shows the Detector class inheritance hierarchy. The child detectors function similarly, each having info and detect functions. The detect functions use the flags discussed in Section 2.3 and run-time polymorphism to discern wether a particle would be detected, Listing 9.

```
13 double ElectromagneticCalorimeter::detect(const Particle &particle) const
14 {
15   if(misdetected()){return 0;}
16   if(!particle.electromagnetic_calorimeter_detectable())
```

```
17    {
18      return 0;
19    }
20          return particle.get_energy();
21  }
22  }
```

Listing 9: Code snippet from ElectromagneticCalorimeter.cpp showing the detect function and how the flags and misdetect function is incorparated.

3 Results

```
1         sub-detector          |             properties
2  ------------------------------------------------------------------
3           tracker             | material : silicon, efficiency : 100%
4  ------------------------------------------------------------------
5   electromagnetic calorimeter | material : argon, efficiency : 100%
6  ------------------------------------------------------------------
7      hadronic calorimeter     | material : steel, efficiency : 90%
8  ------------------------------------------------------------------
9      hadronic calorimeter     | material : iron, efficiency : 95%
10 ------------------------------------------------------------------
11      muon spectrometer       | material : aluminium, efficiency : 100%
12 ------------------------------------------------------------------
```

Listing 10: Expected output for Detector.info() for a detector composing of 1 track, 1 electromagnetic calorimeter, 2 hadronic calorimeter and a muon spectrometer.

```
1  ------------------------------------------------------------------
2  Particle : photon | Expected Energy : 5332 MeV
3  Maximum tracker energy detected : 0 MeV
4  Maximum electromagentic calorimeter energy detected : 5332 MeV
5  Maximum hadronic calorimeter  energy detected : 0 MeV
6  Maximum muon spectrometer energy detected : 0 MeV
7  Particle detected as a photon with energy 5332 MeV
8  ------------------------------------------------------------------
9  ------------------------------------------------------------------
10 Particle : proton | Expected Energy : 1234 MeV
11 Maximum tracker energy detected : 1234 MeV
12 Maximum electromagentic calorimeter energy detected : 0 MeV
13 Maximum hadronic calorimeter  energy detected : 1234 MeV
14 Maximum muon spectrometer energy detected : 0 MeV
15 Particle detected as a proton with energy 1234 MeV
```

```
16 ----------------------------------------------------------------
```

Listing 11: Expected output for Detector when a photon and a proton are passed through it, with the max energy from each sub-detector returned along with a prediction of this outcome based on the energy returned.

```
1 int main()
2 {
3   return 0;
4 }
```

4 Discussion

Options for extending the project are to add more decay modes to the Higgs, for example a ZZ* boson decay along with an accompanying Z boson class and its decays. The decay path chosen could be decided by a random number generator like rand(). This would be fairly easy to implement in the existing code as the decay function already exists, so besides some extra abstraction to keep the decay function clean, very little besides writing the code for the decay itself would be needed.

Another option would be to add a random standard error on the detector reading. A skeleton of a code that would work for this is commented out in the SubDetector class. The code generates a uniform random number and then uses a cumulative distribution function to transform it into an exponential distribution and then to a σ value. If the standard error is then put to the power of this σ value, the result would be akin to a detector having a random standard error. The complication comes with the lack of total abstraction in the sub-detector detect function, where each detectors detect function would have to be changed to add this random standard error to the output. If there was instead a protected member function of SubDetector that took care of calculating the detected energy for all sub-detectors, this would not be the case.